

CS 621 HW3

Ethan Reinhart

October 2025

1

1.1 Question

For the array $A[1, \dots, 8] = [7, 4, 6, 5, 8, 2, 10, 3]$, write down the OPT and $PREV$ array computed by the dynamic-programming algorithm for longest increasing subsequence.

1.2 Solution

$OPT[i]$ = length of the longest increasing subsequence that ends at position i
(and must include $A[i]$)

$PREV[i]$ = index of the previous element in that subsequence

Steps:

1. $i = 1, A[1] = 7$
 $OPT[1] = 1$
 $PREV[1] = 0$
2. $i = 2, A[2] = 4$
 $j = 1, 7 \not< 4$
 $OPT[2] = 1$
 $PREV[2] = 0$
3. $i = 3, A[3] = 6$
 $j = 1, 7 \not< 6$
 $j = 2, 4 < 6 \rightarrow 1 + OPT[2] = 2$
 $OPT[3] = 2$
 $PREV[3] = 2$
4. $i = 4, A[4] = 5$
 $j = 1, 7 \not< 5$

$$\begin{aligned}
j = 2, 4 < 5 &\rightarrow 1 + OPT[2] = 2 \\
j = 3, 6 &\not< 5 \\
OPT[4] &= 2 \\
PREV[4] &= 2
\end{aligned}$$

$$\begin{aligned}
5. \ i = 5, A[5] &= 8 \\
j = 1, 7 < 8 &\rightarrow 1 + OPT[1] = 2 \\
j = 2, 4 < 8 &\rightarrow 1 + OPT[2] = 2 \\
j = 3, 6 < 8 &\rightarrow 1 + OPT[3] = 3 \\
j = 4, 5 < 8 &\rightarrow 1 + OPT[4] = 2 \\
OPT[5] &= 3 \\
PREV[5] &= 3
\end{aligned}$$

$$\begin{aligned}
6. \ i = 6, A[5] &= 2 \\
j = 1, 7 &\not< 2 \\
j = 2, 4 &\not< 2 \\
j = 3, 6 &\not< 2 \\
j = 4, 5 &\not< 2 \\
j = 5, 8 &\not< 2 \\
OPT[6] &= 1 \\
PREV[6] &= 0
\end{aligned}$$

$$\begin{aligned}
7. \ i = 7, A[5] &= 10 \\
j = 1, 7 < 10 &\rightarrow 1 + OPT[1] = 2 \\
j = 2, 4 < 10 &\rightarrow 1 + OPT[2] = 2 \\
j = 3, 6 < 10 &\rightarrow 1 + OPT[3] = 3 \\
j = 4, 5 < 10 &\rightarrow 1 + OPT[4] = 3 \\
j = 5, 8 < 10 &\rightarrow 1 + OPT[5] = 4 \\
j = 6, 2 < 10 &\rightarrow 1 + OPT[6] = 2 \\
OPT[7] &= 4 \\
PREV[7] &= 5
\end{aligned}$$

$$\begin{aligned}
8. \ i = 8, A[5] &= 3 \\
j = 1, 7 &\not< 3 \\
j = 2, 4 &\not< 3 \\
j = 3, 6 &\not< 3 \\
j = 4, 5 &\not< 3 \\
j = 5, 8 &\not< 3 \\
j = 6, 2 < 3 &\rightarrow 1 + OPT[6] = 2 \\
j = 6, 10 &\not< 3 \\
OPT[8] &= 2 \\
PREV[8] &= 6
\end{aligned}$$

Therefore, we end up with

$$OPT = [1, 1, 2, 2, 3, 1, 4, 2]$$

$$PREV = [0, 0, 2, 2, 3, 0, 5, 6]$$

2

2.1 Question

You are given an integer $n > 0$ and k types of bills with denominations $d_1 = 1, d_2, \dots, d_k$ (each d_i is also a positive integer). Provide a dynamic-programming algorithm that computes the minimum number of bills needed to make the amount n . You can assume you have infinitely many bills for each type.

1. Describe what your OPT function should be. (E.g., for text segmentation, the answer is ('let $OPT(i)$ denote whether the prefix $s[1 \dots i]$ can be partitioned into valid words'.))
2. Write down a recursive formula for OPT .
3. Express what the algorithm should return using OPT . (E.g., for text segmentation the answer is: the algorithm should return $OPT(n)$.)
4. Write down an algorithm based on your definitions.

2.2 Solution

1. Let $OPT(i)$ be the minimum number of bills needed to make the amount i , for $0 \leq i \leq n$.
2. (a) Base Case:
 $OPT(0) = 0$
(b) Recursive:
 $OPT(i) = 1 + \min(OPT(i - d_j))$ for $j < i$
3. We want $OPT(n)$ because that is the least amount of bills for n dollars.

4.

```
MIN_BILLS(n, d[1...k]):  
    let OPT[1...n]  
    OPT[0] = 0  
    for i = 1 to n:  
        OPT[i] = +infinity  
        for j = 1 to k:  
            if d[j] <= i:  
                OPT[i] = min(OPT[i], 1 + OPT[i-d[j]])  
    return OPT[n]
```

Complexity: $O(kn)$

3

3.1 Question

Given an array of distinct positive numbers $A = [a_1, a_2, \dots, a_n]$, you are asked to return a sub-array C of A with the maximum size such that for each two numbers x, y in C , either $x = k * y$ or $y = k * x$ for an integer k . Notice that for an array $A = [a_1, a_2, a_3, a_4, a_5]$ as an example, a sub-array could be $C = [a_2, a_3, a_5]$, i.e., if you view A as a set, then C is just a subset of A .

Design a dynamic-programming algorithm to solve the problem. Your algorithm only needs to provide the maximum size; you don't need to provide the actual sub-array achieving the maximum size. Specifically, answer the following:

1. Describe a way to preprocess the array A into another array $B = [b_1, b_2, \dots, b_n]$. Your subsequent dynamic programming solution should be entirely based on this new array B .
2. Describe what your OPT function should be. (E.g., for text segmentation, the answer is: 'let $OPT(i)$ denote whether the prefix $s[1...i]$ can be partitioned into valid words'.)
3. Complete the following recursive formula for OPT :

$$OPT(i) = \begin{cases} \text{if } i = 1 \\ \text{otherwise} \end{cases} \quad (1)$$

4. Express what the algorithm should return using OPT . (E.g., for text segmentation the answer is: the algorithm should return $OPT(n)$.)
5. Write down an algorithm based on your definitions.

3.2 Solution

1. A should be sorted in increasing order so the first element is smallest and then called B .
2. Let $OPT(i)$ be the cardinality of C (the largest valid subset) ending at index i in B .
3. (a) Base Case:
 $OPT(1) = 1$
(b) Recursive:
 $OPT(i) = 1 + \max(OPT(j))$ for $1 \leq j < i$ and $b_i \bmod b_j = 0$
4. $\max(OPT(i))$ for $1 \leq i \leq n$
We want max over all as the last may not be in the longest subset.

```

5.      MAX-SUBGROUP(A, k):
        B = sort(A) # sort from smallest to largest
        n = len(A)
        let OPT[1 ... n]

        OPT[1] = 1

        for i = 2 to n:
            OPT[i] = 1
            for j = 1 to i - 1:
                if B[i] % B[j] == 0: # % is mod operator here
                    OPT[i] = max(OPT[i], 1 + OPT[j])

        maximum = 0
        for i = 1 to n:
            maximum = max(maximum, OPT[i])

        return maximum

```

Complexity: $O(n^2)$ in worst case, $O(n \log n)$ on average.